How Far Can We Go with Practical Function-Level Program Repair?

Jiahong Xiang
Southern University of Science and Technology
Shenzhen, China
xiangjh2022@mail.sustech.edu.cn

Xiaoyang Xu, Fanchu Kong
Southern University of Science and Technology
Shenzhen, China
12112620@mail.sustech.edu.cn
12112822@mail.sustech.edu.cn

Mingyuan Wu
Southern University of Science and Technology
Shenzhen, China
11849319@mail.sustech.edu.cn

Zizheng Zhan
Kwai Inc.
Beijing, China
zhanzizheng@kuaishou.com

Haotian Zhang
Kwai Inc.
Beijing, China
zhanghaotian@kuaishou.com

Yuqun Zhang
Southern University of Science and Technology
Shenzhen, China
zhangyq@sustech.edu.cn

arXiv:2404.12833v2 [cs.SE] 31 Oct 2024

ABSTRACT

Recently, multiple Automated Program Repair (APR) techniques based on Large Language Models (LLMs) have been proposed to enhance the repair performance. While these techniques mainly focus on the single-line or hunk-level repair, they face significant challenges in real-world application due to the limited repair task scope and costly statement-level fault localization. However, the more practical function-level APR, which broadens the scope of APR task to fix entire buggy functions and requires only cost-efficient function-level fault localization, remains underexplored.

In this paper, we conduct the first comprehensive study of LLM-based function-level APR including investigating the effect of the few-shot learning mechanism and the auxiliary repair-relevant information. Specifically, we adopt six widely-studied LLMs and construct a benchmark in both the Defects4J 1.2 and 2.0 datasets. Our study demonstrates that LLMs with zero-shot learning are already powerful function-level APR techniques, while applying the few-shot learning mechanism leads to disparate repair performance. Moreover, we find that directly applying the auxiliary repair-relevant information to LLMs significantly increases function-level repair performance. Inspired by our findings, we propose an LLM-based function-level APR technique, namely SRepair, which adopts a dual-LLM framework to leverage the power of the auxiliary repair-relevant information for advancing the repair performance. The evaluation results demonstrate that SRepair can correctly fix 300 single-function bugs in the Defects4J dataset, largely surpassing all previous APR techniques by at least 85%, without the need for the costly statement-level fault location information. Furthermore, SRepair successfully fixes 32 multi-function bugs in the Defects4J dataset, which is the first time achieved by any APR technique ever to our best knowledge.

1 INTRODUCTION

Fixing software defects costs developers a significant amount of time and effort [34]. To assist developers in reducing the burden of repairing programs, Automated Program Repair (APR) techniques have been proposed to automatically generate potential patches for buggy programs. Specifically, the learning-based APR techniques which incorporate the learning power to advance the repair performance have been increasingly studied in recent years. For instance, many such techniques [30, 39, 49, 56, 81, 82, 89] utilize Neural Machine Translation (NMT) [68] such that APR is modeled as a translation task where the objective is to transform buggy code into correct code.

More recently, Large Language Models (LLMs) have become largely adopted in various downstream software tasks [31, 32, 42, 47] including APR [38, 40, 62, 74, 75, 77, 79] where they have been proven to advance the repair performance [33, 38, 75–77], e.g., the Codex model can fix 32 more bugs than previous APR techniques in the Defects4J 1.2 dataset [76]. Meanwhile, researchers also propose multiple LLM-based APR techniques [40, 71, 74, 77–79] to further enhance the repair performance. For instance, the state-of-the-art LLM-based APR technique ChatRepair [79] employs a conversational repair mechanism and successfully fixes 162 out of 337 bugs in a crafted Defects4J dataset, causing at least 24.6% gain compared with all existing techniques. However, many LLM-based APR techniques are proposed for the single-line or hunk-level program repair by auto-completing a single line [45] or infilling a hunk of code with context [77] respectively. They typically rely on identifying statement-level program faults, i.e., given fault locations [37, 74, 75, 77, 85] or applying statement-level fault localization techniques such as Gzoltar [71]. Nevertheless, it has been widely argued that accurately identifying statement-level faults can be essentially costly, i.e., demanding fine-grained input or strong assumptions [21, 24, 52, 66], thus potentially limiting the real-world applicability of the single-line or hunk-level APR. On the other hand, the LLM-based function-level APR can be potentially more promising, i.e., applying a generative model such as an LLM to auto-regressively generate the entire patched version of the buggy function by prompting the buggy function into the LLM. To illustrate, first, the function-level APR enables a larger scope of the program repair task—it involves not only the single-line and hunk-level repair tasks, but also a more complicated task which repairs multiple discontinuous lines or hunks within a function [65]. Second, identifying function-level faults tends to be more cost-efficient than identifying statement-level faults, thus rendering the function-level APR more practical in real world [44, 48, 55].

While LLM-based function-level APR techniques are more promising, there lacks sufficient study and understanding of them [76], thus potentially hindering the further improved usage of LLMs for APR. Specifically, first, the existing LLM-based APR techniques exhibit significant performance loss for the function-level APR, e.g., incurring a decrease of 33% in ChatRepair and 36.4% in CodexRepair [76, 79] in terms of correctly fixed bugs. Second, the rationale of how such techniques can be affected has not been fully investigated. More specifically, the effectiveness of certain commonly-used mechanism such as the few-shot learning [40, 76, 79], i.e., prompting buggy code and fixed code pair examples that illustrate the function-level APR task and provide repair context for advancing the learning power of models, remains inadequately validated. Additionally, the potential of incorporating the auxiliary repair-relevant information, such as bug reports and trigger tests, remains underexplored [33, 62, 79]. Thus, there is an urgent need to extensively study the LLM-based function-level APR to further enhance the repair performance.

In this paper, we conduct the first comprehensive study on the function-level LLM-based APR including investigating the effect of the few-shot learning and the auxiliary repair-relevant information. Specifically, we adopt six widely-studied LLMs including the state-of-the-art LLMs such as Codex-edit and GPT-3.5-Turbo [33, 79, 85, 86] as the study subjects. We also construct a benchmark containing 522 single-function bugs, i.e., the bugs existing within one single function, in the Defects4J dataset. Typically, we build our repair prompt containing the buggy function along with (1) the buggy code and fixed code pair examples to utilize the few-shot learning mechanism; and (2) the auxiliary repair-relevant information such as trigger tests for the studied LLMs respectively. In this way, the entire patched functions can be auto-generated and then validated with the Defects4J test suite to derive the plausible patches (which pass all the tests). Our evaluation results demonstrate that incorporating the few-shot learning mechanism in the function-level APR actually causes significantly disparate and even negative impacts on the average number of plausible fixes compared with applying the default LLMs only, i.e., from an increase of 10% to a decrease of 49.7% among all studied LLMs. Surprisingly, we find that directly applying trigger tests or error messages for prompting can significantly enhance the repair performance, e.g., 26.7% and 26.1% improvement in terms of the average number of plausible fixes respectively. On the other hand, while statement-level fault location information is shown powerful essential to many APR techniques, only adopting the easier-to-obtain auxiliary repair-relevant information including trigger tests, error messages, and comments altogether for prompting can achieve rather close performance, i.e., causing merely 7.1% gap in terms of the number of plausible fixes. Such a result indicates the potential of replacing the costly statement-level fault location information for the LLM-based function-level APR. In our study, over 10 million patches are generated and validated, consuming more than 8,000 GPU and 100,000 CPU hours. To our best knowledge, this is the largest empirical study of LLM-based APR conducted to date.

Inspired by our findings, we propose an LLM-based function-level APR technique, namely SRepair, which adopts a dual-LLM framework to leverage the power of the auxiliary repair-relevant information for advancing the repair performance. In particular, SRepair first adopts a repair suggestion model which employs the Chain of Thought (CoT) technique [72] to generate natural-language repair suggestions. More specifically, SRepair prompts the LLM with the buggy function and the auxiliary repair-relevant information (i.e., trigger tests, error messages, and comments) to identify the root causes of the bugs and generate repair suggestions in natural language accordingly. SRepair then adopts a patch generation model to auto-generate a patched function with the assistance of the repair suggestions. Our evaluation demonstrates that SRepair can correctly fix a total of 300 single-function bugs in our Defects4J dataset, largely surpassing all previous APR techniques, e.g., 1.59× more than Repilot [74] and 85% more than ChatRepair [79], without the need for the costly statement-level fault location information. Moreover, 128 bugs out of them were not fixed by any of the baseline LLM-based APR techniques adopted in this paper. Surprisingly, SRepair is also capable of repairing 32 multi-function bugs, i.e., bugs existing across multiple functions at the same time, which, to our best knowledge, is the first time achieved by any APR technique ever.

To summarize, this paper makes the following contributions:

- We perform the first ever extensive study on the LLM-based function-level APR with the impact factors on its performance, paving the way for new directions in future research.
- We find that LLMs with zero-shot learning are already powerful function-level APR techniques. We also find that applying auxiliary repair-relevant information can substantially improve the repair performance for all studied LLMs.
- We propose a new LLM-based function-level APR technique, SRepair, which can achieve remarkable repair performance by correctly fixing 300 single-function bugs, largely surpassing the SOTA techniques, i.e., outperforming ChatRepair [79] by 85% and Repilot [74] by 1.59× in the Defects4J dataset. Moreover, SRepair successfully fixes 32 multi-function bugs, which is the first time achieved by any APR technique ever to our best knowledge.

2 BACKGROUND & RELATED WORK

2.1 Large Language Model

Large Language Models (LLMs) contain billions of parameters and are trained on petabyte-scale datasets. They are typically built based on the Transformer architecture [70] comprising an encoder for input processing and a decoder for output token generation. In particular, the decoder-only models as shown in Figure 1a have demonstrated superior text comprehension [61] and code generation capabilities [23]. Thus, they have garnered significant interest of researchers and been widely applied to various downstream tasks in software engineering, e.g., test case generation [42, 47], vulnerability detection [31, 32], and program repair [33, 37, 77]. These models when integrating domain-specific knowledge for specific tasks, are often fine-tuned [67] for further improving their performance. For instance, CodeLlama [63] is fine-tuned based on Llama 2 [69] for generating and discussing code, and Magicoder [7] is fine-tuned with OSS-Instruct [73] to enhance the code generation performance. While fine-tuning requires significant computational resources and specialized datasets [29], simpler prompting strategies like few-shot learning [26] and Chain of Thought (CoT) [72]

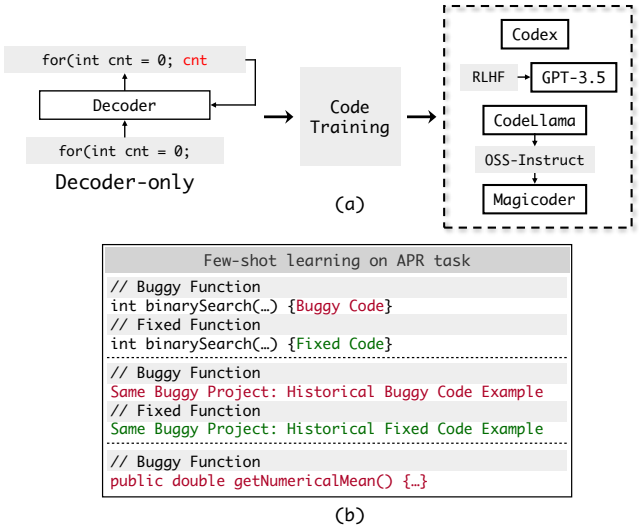


Figure 1: Decoder-only models and few-shot learning on APR

which have also been shown effective are much less costly and thus have been increasingly adopted [20, 84, 88]. Figure 1b illustrates how the few-shot learning mechanism is applied for APR. Firstly, the APR task-related examples such as the buggy and fixed code pair of the function `binarySearch` are incorporated into the prompt. Note that the example selection varies among different techniques, e.g., manually crafting examples like `binarySearch` [76] and choosing examples of historical bug fixes within the same buggy project [76, 79]. Next, the target buggy function to be fixed, e.g., `getNumbericalMean()` in the Math-2 bug [14], is also added to the prompt. At last, the resulting prompt is fed to the model to generate patches for the target buggy function. To summarize, the purpose of the few-shot learning mechanism is to enable the model to learn how to handle specific tasks through the examples. However, while multiple LLM-based APR techniques have already incorporated the few-shot learning mechanism [40, 76, 79], its impacts and characteristics remain unexplored.

2.2 Automated Program Repair

Automated Program Repair (APR) techniques [33, 39, 40, 46, 56, 74–77, 81], designed to aid developers in fixing bugs by automatically generating patches, typically follow the Generate-and-Validate (G&V) paradigm [54]. In particular, an APR process refers to locating program faults, generating patches for the buggy locations, and validating such patches against a test suite to determine their plausibility (i.e., whether they could pass all tests). Eventually, these resulting plausible patches are manually reviewed to select the correct fix for the target fault. Notably, the trigger tests in the patch-validating test suite are manually created by developers. During the execution of trigger tests, the unit testing framework, e.g., JUnit [5], can be used to provide the corresponding error messages.

Among the APR techniques, the learning-based techniques [39, 49, 56, 74, 77, 79, 81] that utilize deep learning techniques have recently achieved remarkable performance. Specifically, many such techniques widely adopt the Neural Machine Translation (NMT) [68] techniques which convert APR into a translation task to transform buggy code into correct code. They typically leverage the power of the NMT models through training on extensive datasets containing millions of buggy and fixed code pairs. However, such techniques are highly costly when building well-constructed datasets of buggy and patch code pairs [77] and specific context representation for the NMT models [39]. More recently, Large Language Models (LLMs) have become increasingly adopted in various downstream software tasks including APR. In particular, directly applying models like Codex can already outperform all previous APR techniques [76]. Meanwhile, multiple LLM-based APR techniques have been proposed to further enhance the repair performance. For instance, AlphaRepair [77] applies the pre-trained CodeBERT model with the "cloze-style" APR, i.e., removing the buggy code tokens and applying the LLM to generate the correct ones. Similar as AlphaRepair in adopting the cloze-style repair paradigm, Repilot [74] focuses on synthesizing compilable patches, utilizing the Language Server Protocol to prune infeasible tokens and proactively complete tokens as suggested by the LLM. FitRepair [75] combines LLMs with domain-specific fine-tuning and prompting strategies, fully automating the plastic surgery hypothesis, i.e., the code ingredients to fix bugs usually already exist within the same project. ChatRepair [79] utilizes the conversational repair mechanism based on GPT-3.5 and successfully fixes 162 out of 337 bugs in the Defects4J dataset with the assistance of the rich information from original bug-exposing tests. Fan et al. [33] conduct a study to investigate whether APR techniques can correct program errors generated by LLMs, particularly in complex tasks like the LeetCode contests. Another study [76] employs the few-shot learning mechanism and recognizes the ineffectiveness of simply feeding LLMs with only buggy functions as they are not pre-trained for APR. To address this, they create a prompt containing two pairs of buggy and fixed code examples: one manually crafted, and another from the same project or dataset. Then they include the buggy function to be fixed in this prompt, thus activating the function-level APR by providing such a prompt to LLMs. However, their claim that LLMs cannot be directly applied to the function-level APR and the effectiveness of employing the few-shot learning mechanism has not been fully investigated.

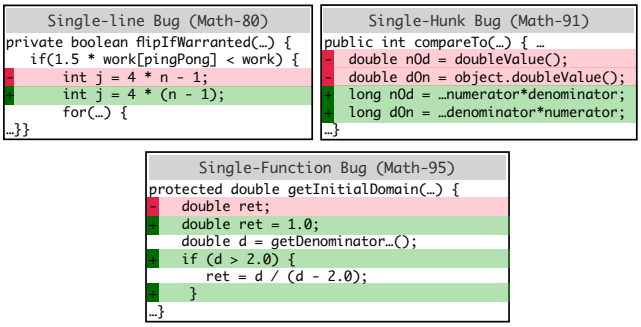


Figure 2: Bug examples existing in a single line, hunk, or function

The existing LLM-based APR techniques mainly focus on repairing single-line or single-hunk bugs [74, 75, 77, 85], as illustrated

in Figure 2. Specifically, the single-line bug Math-80 [15] is contained within a single line of the function `flipIfWarranted` where fixing this line alone can resolve the bug. Such a single-line bug can be fixed by APR techniques focusing on the line-level repair like AlphaRepair [77] given accurate fault locations. Meanwhile, the single-hunk bug Math-91 [16] is contained in a continuous section of code where two contiguous buggy lines are replaced in the fixed version. This kind of bugs can be fixed by the hunk-level APR techniques like Repilot [74] requiring multiple accurate statement-level fault locations. Note that single-line and single-hunk bugs can be considered as part of single-function bugs. On the other hand, the bugs existing in multiple discontinuous sections/lines within a function and requiring simultaneous edits on them for a fix are also referred to as single-function bugs. For instance, fixing the Math-95 bug [17] requires editing three discontinuous lines simultaneously. It can be easily derived that fixing single-function bugs poses a greater challenge for APR techniques, e.g., the state-of-the-art APR techniques like ChatRepair and CodexRepair incur a performance decrease of 33% and 36.4% [79] in terms of the number of correctly fixed bugs for the function-level APR respectively.

As mentioned, single-function bugs actually include single-hunk and single-line bugs as subset, i.e., enabling a larger scope of repair tasks. Moreover, locating function-level bugs tends to be cheaper than locating line-level or hunk-level bugs [21, 48, 55], thus making the function-level APR more practical. Therefore, we consider developing the function-level APR techniques rather promising and worthy being sufficiently investigated.

3 EMPIRICAL STUDY

3.1 Study Setup

3.1.1 Study Subjects. We utilize six distinct LLMs as our study subjects, encompassing the widely used state-of-the-art Codex-edit and GPT-3.5-Turbo [33, 76, 79, 86], along with four advanced open-source code LLMs including CodeLlama 7B, 13B, and 34B (over 500k downloads on Hugging Face within one month [6]), and Magicoder 7B (over 1000 stars on GitHub [7]). Specifically, we adopt code-davinci-edit-001 [8], gpt-3.5-turbo-1106 [18], and the CodeLlama-Instruct series [6] models as the versions of as Codex-edit, GPT-3.5-Turbo, and CodeLlama respectively since they have conducted the instruction fine-tuning [63] and can better follow the APR prompt instruction. We also employ the MagicoderS-CL [7] model as the version of Magicoder. Due to the page limit, the LLM configuration details are presented in our *GitHub* page [1].

3.1.2 Dataset. We construct our benchmark using both the versions 1.2 and 2.0 of the Defects4J dataset [41] which is the most widely used APR dataset [39, 57, 76] with a collection of a total of 835 real-world bugs extracted from open-source Java projects, comprising both buggy and fixed versions of the source code. Notably, we bound our study scope within 522 single-function bugs including 276 single-hunk ("SH") bugs and 158 single-line ("SL") bugs as shown in Table 1. It should be noted that our collected single-function bugs already include all the single-line and single-hunk bugs studied in previous works [74–77, 79].

3.1.3 Evaluation Metrics. To assess the repair performance, we follow the standard practice [35, 43, 60], to utilize the plausible patches that pass all test cases as our major evaluation metric. In particular, those test cases include the trigger tests in the Defects4J dataset designed to expose bugs and relevant tests which can load the classes associated with the buggy functions.

Table 1: Statistics of the Dataset

Dataset	Project	# Bugs	SH Bugs	SL Bugs
Defects4j 1.2	Chart	16	12	9
	Closure	105	59	26
	Lang	42	23	13
	Math	74	35	23
	Mockito	24	12	7
	Time	16	6	3
Defects4j 2.0	Cli	28	13	6
	Codec	11	9	8
	Collections	1	1	1
	Compress	36	16	5
	Csv	12	7	4
	Gson	9	5	4
	JacksonCore	13	9	5
	JacksonDatabind	67	26	15
	JacksonXml	5	1	1
	Jsoup	53	38	27
	JxPath	10	4	1
Overall		522	276	158

3.2 Research Questions

We investigate the following research questions for extensively studying the function-level APR along with the factors which can impact its effectiveness.

- **RQ1:** *How does the LLM-based function-level APR perform under the zero-shot and few-shot learning setups?* For this RQ, we attempt to investigate the performance of the LLM-based function-level APR under the default zero-shot learning and explore the performance impact from adopting few-shot learning.
- **RQ2:** *How do different auxiliary repair-relevant information affect the LLM-based function-level repair performance*? For this RQ, we attempt to study the impact from different auxiliary repair-relevant information including bug reports, trigger tests, etc., on the function-level repair performance.

3.3 Implementation

We obtain the model from Hugging Face [2] and access Codex-edit and GPT-3.5-Turbo through API [3] provided by OpenAI. Our default setting for patch generation uses the nucleus sampling with top p = 0.9, temperature = 0.8 and 200 samples per bug following prior works [28, 62, 76]. Patches are generated on servers with 128-core 2.6GHz AMD EPYC™ ROME 7H12 CPU, 512 GiB RAM and eight NVIDIA A100 80GB GPUs, running Ubuntu 20.04.6 LTS.

3.3.1 APR input prompt setup. Following prior studies [62, 76], we set the prompt for the LLM utilized in the APR task, as illustrated in Figure 3 to enable the function-level APR. Specifically, we begin with a description of the APR task as '`Provide a fix for the buggy function`'. Next, we incorporate the buggy code and fixed code pair examples from the few-shot learning mechanism or

```
// Provide a fix for the buggy function
{Buggy code and fixed code pair examples} OR
{Auxiliary repair-relevant Information}
// Buggy Function
public double getNumericalMean() {
        return (double) (getSampleSize() *
 getNumberOfSuccesses()) / (double) getPopulationSize();}
// Fixed Function
```

Figure 3: The input prompt for the function-level APR of the Math-2 bug

Table 2: K-shot learning settings and abbreviations

K	Example Type	Abbreviation
0	N.A.	K_0*(Basic)*
1	Crafted Example	K_1*(CE)*
1	Project Example	K_1*(PE)*
2	Crafted Example & Project Example	K_2*(CE, PE)*
2	Project Example & Project Example	K_2*(PE, PE)*

the auxiliary repair-relevant information into the prompt. Subsequently, we use '`Buggy Function`' in conjunction with the buggy code, e.g., the Math-2 buggy function `getNumericalMean` [14] in Figure 3, to prompt LLMs with the buggy function to be fixed. Finally, we apply '`Fixed Function`' to guide the LLM in generating a patched function. Notably, we employ the zero-shot learning approach as the default baseline in our study, i.e., adopting no auxiliary repair-relevant information or buggy-fixed code pair examples for prompting.

3.3.2 K-shot learning setups. Table 2 presents our k-shot learning setups. Specifically, we set the zero-shot learning approach, i.e., adopting no pairs of buggy code and fixed code examples (K=0), as our basic setup denoted as K_0*(Basic)*. Moreover, we follow prior work [76] to form our buggy and fixed code pair examples via manually crafted examples (*CE*), i.e., `binarySearch` in Figure 1b and chosen historical bug fix examples within the same buggy project (*PE*). We thus form our k-shot learning setup variants as K_1*(PE)* with only one chosen historical bug fix example from the same project, K_1*(CE)* with only one manually crafted example, and K_2*(CE, PE)* with one manually crafted example and one chosen historical bug fix example from the same project. We also form K_2*(PE, PE)* with two chosen historical bug fix examples from the same project following the implementation of the prior work for selecting multiple examples [19]. Note that while it is possible for more setup variants, e.g., with more manually crafted examples and chosen historical bug fix examples from the same project, we generally follow the setup of prior work [76] for fair performance comparison and cost-efficient evaluations.

3.3.3 Collecting auxiliary repair-relevant information. In this study, we refer to the auxiliary repair-relevant information as the bug report and project-specific information from the target buggy project following prior works [46, 64, 79, 81, 83], as the Math-2 bug shown in Figure 4. Specifically, the bug reports are collected from the official issue links of the Defects4J [4] repository. More specifically, following prior works [27, 87], we divide a bug report into two parts, as

(a) Bug Report

Issue Title

Issue 1021: HypergeometricDistribution.sample suffers from integer overflow

Issue Description

"Hi, I have an application which broke when ported from commons math 2.2 to 3.2. It looks like the HypergeometricDistribution.sample() method doesn't work as well as it used to with large integer values, the example code below should return a sample between 0 and 50, but usually returns -50…"

(b) Project-specific Information

Trigger Test

```
public void testMath1021() {
…
for (…) {
  Assert.assertTrue(0 <= sample);
  Assert.assertTrue(sample <= n);
}}
```

Error Message

AssertionFailedError: sample=-50 at HypergeometricDistributionTest.java:297

Comment

/* For population size {@code N}, number of successes {@code m}, and sample*size {@code n}, the mean is {@code n*m/N}. */

Figure 4: The bug report and project-specific information in the Math-2 bug

illustrated in Figure 4a. One is the issue title with averagely around 12 tokens to summarize the type and the cause of the bug (e.g., *"Issue 1021: ... suffers from integer overflow"*). The other is the issue description with averagely 234 tokens which provides detailed conditions, error messages, and reproduction steps, etc. For instance, the issue description in the Math-2 bug report provides a detailed description of the buggy method `HypergeometricDistribution.sample()` with the trigger conditions, i.e., *"with large integer values"*.

Furthermore, we automatically extract the project-specific information from the buggy project, as the Math-2 bug in Figure 4b following the prior works [62, 79, 81]. Specifically, we first build all the buggy projects and automatically extract all the trigger tests and buggy function comments. Then, for each bug, we execute the trigger tests and capture the error messages generated by the unit test framework, such as Junit [5]. Notably, among all 522 single-function bugs in the Defects4J dataset, only 10 miss reports and 2 miss comments. We then leave such auxiliary repair-relevant information empty in our study.

For evaluating the impact from the auxiliary repair-relevant information, we form eight different setups. Specifically, for bug report-relevant information, we form three setups: *BR(IT)* with the issue title only, *BR(ID)* with the issue description only, and *BR(ALL)* with the whole bug report. For the project-specific information, we form four setups: *PI(TT)* with the trigger test only, *PI(EM)* with the error message only, *PI(BC)* with the buggy comment only, and *PI(ALL)* with all such information.

3.4 Result Analysis

3.4.1 RQ1: the function-level repair performance. Table 3 presents the function-level APR results in terms of the number of plausible fixes. In general, we observe that K_0*(Basic)* achieves the overall optimal plausible fix results, i.e., 180 average plausible fixes out of our collected 522 single-function bugs, outperforming all the rest setups by at least 10.4%. Such a result indicates that LLMs themselves (with zero-shot learning) are already powerful function-level APR techniques.

Table 3: APR results under different few-shot learning setups

Settings	Codex-edit	GPT-3.5-Turbo	CodeLlama			Magicoder	*Average Plausible Fixes*
			7B	13B	34B		
K_0(Basic)	**174**	175	192	179	160	**199**	**180**
K_1(CE)	103	138	180	185	**176**	112	149
K_1(PE)	109	174	**194**	**193**	153	157	163
K_2(CE, PE)	138	166	175	189	125	100	149
K_2(PE, PE)	165	**187**	167	189	128	121	160

Finding 1: LLMs with zero-shot learning are already powerful function-level APR techniques.

Interestingly, we can further observe that applying the few-shot learning mechanism leads to quite disparate plausible fix results across LLMs. For instance, compared with *K_0(Basic)*, while CodeLlama 34B shows a 10% (176 vs. 160) improvement in *K_1(CE)*, Magicoder shows a 49.7% (100 vs. 199) decline in terms of the number of plausible fixes.

Finding 2: Applying the few-shot learning mechanism in the function-level APR leads to disparate plausible fix results across LLMs.

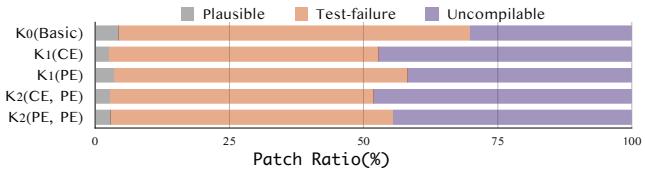
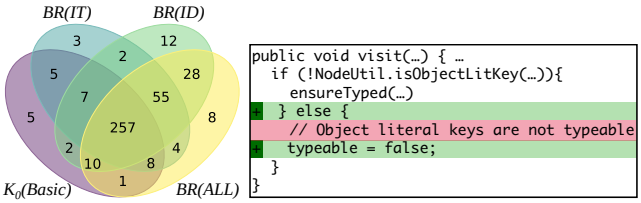


Figure 5: Patch status averaged across all models under different few-shot learning setups

Furthermore, we present the distribution of plausible, test-failure, and uncompilable patches given the identical total number of generated patches across different LLMs under all setups in Figure 5. Note that the test-failure patches can be successfully compiled but fail one or more tests in our Defects4J dataset. Interestingly, we can find that *K_0(Basic)* achieves the best plausible patch rate 4.3% and the lowest uncompilable patch rate 30.2% among all the k-shot setups, while applying the few-shot learning generates more uncompilable patches, i.e., ranging from 38.4% to 59.6% more than *K_0(Basic)*.

Finding 3: Applying the few-shot learning mechanism may generate more uncompilable patches than the zero-shot learning mechanism.

3.4.2 RQ2: performance impact from the auxiliary repair-relevant information. Noticing that since applying zero-shot learning achieves the optimal repair performance among all the k-shot learning techniques as mentioned, we also adopt zero-shot learning in our auxiliary repair-relevant information evaluations and *K_0(Basic)* as a baseline for a fair performance comparison. Table 4 presents the



(a) Venn diagram **(b) Closure-66 bug**

Figure 6: The Venn diagram of plausible fixes over different setups and the bug Closure-66 which can only be fixed in *K_0(Basic)*

APR results under different auxiliary repair-relevant information setups. We observe that using bug report-relevant setups significantly enhances the repair performance of all models, i.e., the number of average plausible fixes increases from 180 in *K_0(Basic)* to 238 in *BR(IT)*, 270 in *BR(ID)*, and 273 in *BR(ALL)*. On the other hand, while *BR(ALL)* achieves the optimal result, Figure 6a also shows that it misses fixing 19 bugs which can be fixed in *K_0(Basic)*. More specifically, we can observe that five bugs can only be fixed in *K_0(Basic)* other than all the rest setups, such as Closure-66 [11] in Figure 6b. We find that to fix such a bug, a simple branch condition must be added to set `typeable` false. In *K_0(Basic)*, four models successfully fix this bug. However, in *BR(IT)*, *BR(ID)*, and *BR(ALL)*, the focus is incorrectly placed on the issue described in the bug report, leading to inappropriate code logic changes. Consequently, none of the six models are able to fix the bug.

Finding 4: While applying the bug report-relevant information significantly enhances the function-level repair performance, it still misses fixing certain bugs which can be fixed by the baseline technique.

We also attempt to investigate the performance impact from the project-specific information on the LLM-based function-level APR. Table 4 shows that using project-specific information setups leads to an increase for all models, i.e., the average number of plausible fixes rises from 180 in *K_0(Basic)* to 185 in *PI(BC)*, 227 in *PI(EM)*, 228 in *PI(TT)*. Notably, *PI(ALL)* achieves an optimal average of 254 plausible fixes, indicating the potential of leveraging as much auxiliary repair-relevant information as possible for enhancing the function-level repair performance. Interestingly, unlike adopting the bug report-relevant information, all the bugs plausibly fixed in *K_0(Basic)* can also be fixed by adopting the project-specific information.

Table 4: APR result in different auxiliary repair-relevant information settings

Sources	Settings	Codex-edit	GPT-3.5-Turbo	CodeLlama			Magicoder	*Average Plausible Fixes*
				7B	13B	34B		
N.A.	***K_0(Basic)***	174	175	192	179	160	199	180
Bug Report Information	***BR(IT)***	265	233	234	221	221	251	238
	BR(ID)	281	**286**	261	**264**	248	**279**	270
	BR(ALL)	**301**	285	**275**	260	**255**	260	**273**
Project-specific Information	***PI(BC)***	186	185	187	191	169	194	185
	PI(EM)	217	226	239	225	217	240	227
	PI(TT)	239	247	227	221	201	235	228
	PI(ALL)	**264**	**273**	**249**	**247**	**236**	**254**	**254**

Table 5: APR results with fault location information

Sources	Settings	w/o †FL	w/ †FL	Improvement
N.A.	***K_0(Basic)***	180	217	**20.6%**
Bug Report Information	***BR(IT)***	238	262	**10.1%**
	BR(ID)	270	289	7.0%
	BR(ALL)	273	291	6.6%
Project-specific Information	***PI(BC)***	185	217	**17.3%**
	PI(EM)	227	257	13.2%
	PI(TT)	228	246	7.9%
	PI(ALL)	254	272	7.1%

†FL refers to fault location information.

Finding 5: Directly adopting trigger tests, error messages, and comments from buggy projects can also effectively advance the function-level repair performance.

We further evaluate the performance impact and necessity of the statement-level fault location information in the function-level APR. We utilize the ground-truth statement-level fault location information following previous work [62] by labeling the corresponding buggy line with `/*bug is here*/`. Specifically, the ground-truth fault locations are provided by the official Defects4J GitHub Repository [12]. To investigate the impact of the statement-level fault location information on the function-level APR, we calculate the average number of plausible fixes generated by various models across different auxiliary repair-relevant information setups.

```
private boolean inferTemplatedTypesForCall(…) {
  …
- Map<TemplateType, JSType> inferred = /* bug is here */
-     inferTemplateTypesFromParameters(fnType, n); /* bug is here */
+ Map<TemplateType, JSType> inferred = Maps.filterKeys(…);
  TemplateTypeReplacer replacer = …
  return replacer.madeChanges;
}
```

Figure 7: Statement-level fault location information misleads LLM, preventing the repair of the Closure-112 bug

From Table 5, we can observe that while applying the statement-level fault location information enhances the repair performance, the extent of this improvement can be potentially compromised with the token number increase of the auxiliary repair-relevant information. For instance, while $K_0(Basic)_{FL}$ achieves a performance improvement of 20.6% compared to $K_0(Basic)$, such an improvement shrinks to 6.6% comparing $BR(ALL)_{FL}$ to $BR(ALL)$ with averagely 246 tokens and 7.1% comparing $PI(ALL)_{FL}$ to $PI(ALL)$ with averagely 396 tokens. Moreover, we find that 14 bugs that are originally fixable without fault location information cannot be plausibly fixed across all setups and models when using fault location information. For instance in the Closure-112 bug [10] shown in Figure 6b which demands multiple edits, a correct fix is achieved if the model reads the entire method, thus comprehending the necessity of adding `Maps.filterKeys` to check if each key (of the `TemplateType` type) exists in the key collection. However, with the fault location information, the attention of the model becomes disturbed, consequently over-focusing on the `Map<TemplateType, JSType> inferred` code block and making extensive but ineffective modifications.

Finding 6: The statement-level fault location information effectively enhances the repair performance. As the token number of auxiliary repair-relevant information increases, the extent of the improvement can be potentially compromised.

4 DISCUSSION

4.1 Bug report

While the bug reports associated with carefully evaluated projects like Defects4J are generally of high quality where their effectiveness can be shown in our evaluation results, they nonetheless include instances of inaccuracies [46]. Specifically, real-world bug reporting is filled with a significant volume of reports that are invalid, irreproducible, incomplete, or outright misleading [22, 25, 51]. Moreover, the process of generating bug reports is manual and labor-intensive, in contrast to the APR techniques seeking to rectify software bugs autonomously, eliminating the need for human intervention [36]. Consequently, relying on bug reports for providing auxiliary repair-relevant information to advance the function-level APR may be inappropriate and impractical, especially when dealing with unknown faults. On the contrary, trigger tests [53, 80, 81] precisely identify the root cause of faults. Error messages [59, 81] can be automatically obtained from test outputs and reveal the fault-triggering boundary conditions. Comments provide function descriptions added by developers [62]. These sources of information are more precise and cost-efficient compared to bug reports. Therefore, we recommend the utilization of project-specific information in LLM-based APR techniques to further improve repair performance.

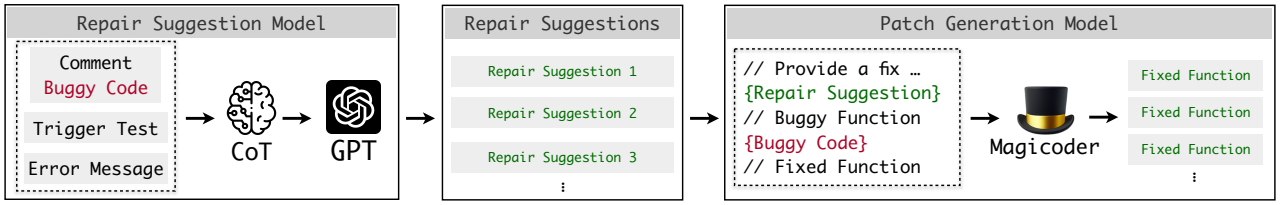


Figure 8: The SRepair framework

4.2 Models for APR

Although the CodeLlama models have gained a number of plausible fixes in our study, we do observe abnormal behaviors of CodeLlama-based models in the patch generation of the function-level APR. When applied to the Java-based Defects4J dataset, CodeLlama models frequently generate patches with '[PYTHON]' tags and Python code, e.g., producing 188,113 such patches in the CodeLlama 34B model. This issue was not prevalent in other models. Hence, we advocate using the high-performing GPT-3.5-Turbo and the open-source Magicoder models, both of which have shown superior capabilities in the APR task.

5 APPROACH

By far, we have demonstrated the power of adopting the auxiliary repair-relevant information in the function-level LLM-based APR, i.e., including such information in the repair prompt along with the buggy function under zero-shot learning. In this section, to further leverage the potential of the auxiliary repair-relevant information, we construct a novel function-level APR technique SRepair (referring to **S**uggestion **Repair**), which adopts a dual-LLM framework for advancing the repair performance.

5.1 SRepair Framework

Our Dual-LLM framework is shown in Figure 8 where SRepair first adopts a repair suggestion model which utilizes the learning power of LLM by comprehensively analyzing the auxiliary repair-relevant information via the Chain of Thought (CoT) technique [72]. Then it provides repair suggestions in natural language. Next, SRepair adopts a patch generation model which exhibits its code generation capabilities by generating the entire patched function following the repair suggestions. More specifically, we enable the CoT technique by prompting the LLM to first analyze the buggy function and project-specific information, then identify the root cause of the bug, and finally generate repair suggestions in natural language. For instance, as shown in Figure 9, the repair suggestion model first identifies the root cause of the Cli-26 bug [9]: '*are not being reset after creating an Option*', and then generates the correct repair suggestion, '*use a try-finally block*'. Finally, such a suggestion is fed to the patch generation model for generating the patched functions.

5.2 Evaluation

5.2.1 Dataset. We use the widely studied repair benchmark of Defects4J [41] and QuixBugs [50]. Specifically, to extensively leverage SRepair's ability in the function-level APR, we include all function-level bugs from Defects4J 1.2 and 2.0, thereby forming a dataset

Table 6: Statistics of SRepair Dataset

Dataset	Project	# Bugs	SF Bugs	MF Bugs
Defects4j 1.2	Chart	25	16	9
	Closure	140	105	35
	Lang	56	42	14
	Math	102	74	28
	Mockito	30	24	6
	Time	22	16	6
Defects4j 2.0	Cli	30	28	2
	Codec	13	11	2
	Collections	2	1	1
	Compress	40	36	4
	Csv	13	12	1
	Gson	12	9	3
	JacksonCore	18	13	5
	JacksonDatabind	85	67	18
	JacksonXml	5	5	0
	Jsoup	58	53	5
	JxPath	14	10	4
Overall		665	522	143

that comprises 522 single-function (SF) bugs and an additional 143 multi-function (MF) bugs, i.e., the bugs existing in multiple functions and requiring simultaneous edits on them for a fix, as shown in Table 6. Additionally, we also evaluate on the QuixBugs dataset which is made up of 40 function-level buggy and fixed versions of classic programming problems in both Python and Java.

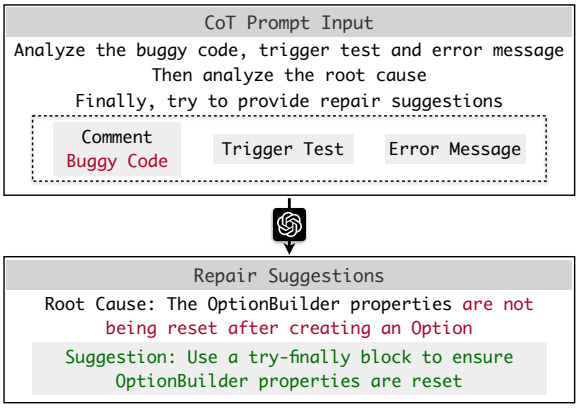


Figure 9: Chain of Thought example of Cli-26 Bug

Table 7: Single-function APR result of SRepair

Datasets	Project	Plausible Fixes						Correct Fixes				
		PI(ALL)		SRepair Variant				AlphaRepair	Repilot	FitRepair	ChatRepair	SRepair_{500}
		GPT-3.5-Turbo	Magicoder	SRepair_{2M}	SRepair_{2M+FL}	SRepair_{200}	SRepair_{500}					
Defects4J1.2	Chart	12	11	14	14	14	**14**	9	6	8	**15**	13
	Closure	40	30	39	49	48	**56**	23	22	29	37	**47**
	Lang	19	25	27	29	29	**32**	13	15	19	21	**26**
	Math	48	43	50	48	47	**55**	21	21	24	32	**42**
	Mockito	8	8	12	9	12	**12**	5	0	6	6	**11**
	Time	7	5	5	6	6	**7**	3	2	3	3	**7**
Defects4J2.0	Cli	16	13	16	17	17	**19**	5	6	6	5	**18**
	Codec	8	5	8	8	8	**11**	6	6	5	8	**11**
	Collections	0	1	0	1	1	**1**	0	1	1	0	**1**
	Compress	21	22	21	24	26	**28**	1	3	2	2	**21**
	Csv	10	9	10	9	10	**11**	1	3	2	3	**11**
	Gson	6	8	7	7	7	**9**	2	1	1	3	**8**
	JacksonCore	9	6	9	7	9	**10**	3	3	3	3	**10**
	JacksonDatabind	30	28	39	38	39	**45**	8	8	10	9	**33**
	JacksonXml	3	1	1	3	1	**3**	0	0	0	1	**2**
	Jsoup	34	35	33	35	35	**39**	9	18	13	14	**35**
	JxPath	2	4	4	5	4	**5**	1	1	1	0	**4**
D4J 1.2 Total		134	122	147	155	156	**176**	74	66	89	114	**146**
D4J 2.0 Total		139	132	148	154	157	**181**	36	50	44	48	**154**
Overall		273	254	295	309	313	**357**	110	116	133	162	**300**

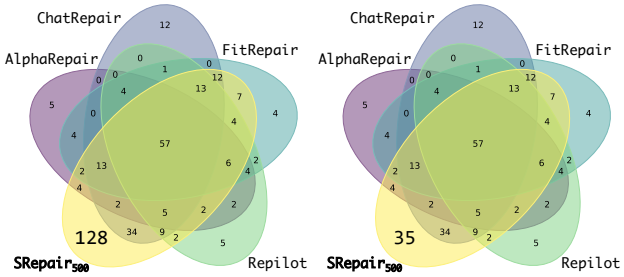
5.2.2 Implementation. In the SRepair implementation, GPT-3.5-Turbo acts as the repair suggestion model due to its superior analytical, coding, and natural language generation abilities, especially in *PI(ALL)*. Magicoder is adopted as the patch generation model due to its cost-effectiveness and competent code generation ability. Notably, for each repair suggestion, SRepair generates 5 patched functions via the patch generation model. We set the sample size of SRepair 200 (denoted as SRepair_{200}) for comparing with previous APR results in our study section and 500 (denoted as SRepair_{500}) for fair comparisons with previous APR techniques [56, 74, 75, 77]. Similar to prior work [39, 49, 77], we additionally add an end-to-end time limit of 5 hours to fix one bug. Moreover, to better repair the Python bugs in QuixBugs, we replace the Java comment symbol '//' in the input APR prompt with the Python comment symbol '#'. It should be noted that SRepair does not require statement-level fault location information. For the rest setups, we follow our study section. Due to the page limit, we show the experimental results under different configurations and costs of SRepair in our GitHub page [1].

5.2.3 Evaluation Metrics. Following our study section, we utilize plausible patches to reflect the repair performance. Furthermore, following standard practices in the APR research, we manually inspect each plausible patch for semantic equivalency [54, 56, 58, 76, 77] to determine the correct patches. Due to the intensive manual efforts involved in patch inspection, we conduct a cross-validation with three authors in order to filter out the correct patches generated by SRepair_{500}.

5.2.4 Compared Techniques. We adopt four recent SOTA LLM-based APR techniques: AlphaRepair [77], Repilot [74], FitRepair [75], and ChatRepair [79]. We also adopt GPT-3.5-Turbo$_{PI(ALL)}$ and Magicoder$_{PI(ALL)}$ as baselines with the same auxiliary repair-relevant information and models used in SRepair for studying the effectiveness of our Dual-LLM CoT framework. We also form two SRepair_{200} variants: SRepair_{2M} with Dual-LLM only, i.e., directly generating repair suggestions without CoT, and SRepair_{2M+FL} with additional statement-level fault location information for comparison.

Table 8: Correct fixes on QuixBugs datasets

QuixBugs	SRepair_{500}	SRepair_{200}	ChatRepair	AlphaRepair
Python	**40**	40	40	27
Java	**40**	40	40	28



(a) Single-function dataset **(b) Studied baselines dataset**

Figure 10: Bug fixes Venn diagram of SRepair_{500} with studied baselines

5.2.5 Result analysis. Table 7 presents the APR results for single-function bugs in the Defects4J dataset. Surprisingly, we find that SRepair_{500} outperforms all previous LLM-based APR techniques by at least 85%. Specifically, we can observe that 68.4% of single-function bugs (357) in Defects4J can be plausibly fixed, and even 57.5% of bugs (300) can be correctly fixed by SRepair_{500}. Such surprising results indicate that SRepair is capable of fixing a significant number of real-world complicated bugs in the function-level APR. Notably, repairing 300 single-function bugs with SRepair costs only \$8.6, averaging \$0.029 per correct fix, demonstrating its efficiency as an LLM-based APR technique.

Moreover, as in Figure 10a, SRepair$_{500}$ correctly fixes 128 out of 522 single-function bugs which cannot be fixed by any of the baseline LLM-based APR techniques adopted in this paper. Interestingly, Figure 10b shows that SRepair$_{500}$ also significantly outperforms the state-of-the-art APR baselines, correctly fixing 35 unique bugs that all other baselines failed to fix in their studied bugs. Such a result indicates that SRepair not only expands the repair task scope to the more practical function-level APR but also achieves remarkable repair performance without the need for statement-level fault location information. Table 8 shows that SRepair$_{500}$ successfully fixes all bugs in the QuixBugs dataset, indicating its superior capability for diverse programming languages.

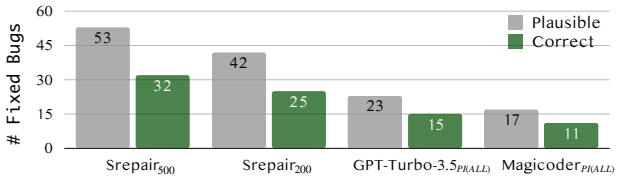


Figure 11: The APR results of the multi-function bugs in the Defects4J dataset

We also evaluate how SRepair repairs complicated multi-function bugs shown in Figure 11 where we find that SRepair$_{500}$ (53 plausible fixes and 32 correct fixes) and SRepair$_{200}$ (42 plausible fixes and 25 correct fixes) both largely outperform GPT-3.5-Turbo$_{PI(ALL)}$ and Magicoder$_{PI(ALL)}$. Interestingly, as shown in Figure 12 where Functions 1 and 2 require information from successfully running Function 3 to determine if they should execute subsequent statements. This poses a significant challenge for APR techniques, as they need to simultaneously alter the return type of Function 3 to boolean and adapt the function calls in Functions 1 and 2. SRepair successfully identifies such a complex function call and generates the correct fix, indicating the power of SRepair on complicated multi-function faults, which, to our best knowledge, is the first time achieved by any APR technique ever.

We further find that SRepair$_{2M}$ outperforms both GPT-3.5-Turbo$_{PI(ALL)}$ and Magicoder$_{PI(ALL)}$ by 8.1% and 16.1% in terms of the number of plausible fixes respectively. Furthermore, leveraging CoT technique achieves even better result (313 plausible fixes) than incorporating statement-level fault localization information (309 plausible fixes). Such results indicate the effectiveness of our Dual-LLM framework and CoT mechanism in SRepair.

```
                    Function 1 (addDelegatingCreator)
public void addDelegatingCreator(…) {…  | public void addDelegatingCreator(…) {…
- verifyNonDup(…);                      | + if (verifyNonDup(…)){
    _arrayDelegateArgs = injectables;   |     _arrayDelegateArgs = injectables;
  …                                     | + } …
}                                       | }
                    Function 2 (addPropertyCreator)
public void addPropertyCreator(…) {     | public void addPropertyCreator(…) {
- verifyNonDup(…);                      | + if (verifyNonDup(…)){
    …                                   |     …
}                                       | + }
                                        | }
                    Function 3 (verifyNonDup)
- protected void verifyNonDup(…) {      | + protected boolean verifyNonDup(…) {
  …                                     |   …
        if (!explicit) {                |         if (!explicit) {
-         return;                       | +         return false;
        }…                              |         } …
}                                       | + return true;
                                        | }
```

Figure 12: The multi-function bug JacksonDatabind-69 [13]

6 THREATS TO VALIDITY

Threats to internal validity. One potential threat arises from our manual validation process, which differentiates between plausible patches and those that are semantically correct. To address this concern, three authors cross-validated the plausible patches of SRepair$_{500}$ by comparing them to those created by developers (the plausible patches generated by other techniques are mostly subsets).

Another threat is the potential for data leakage if the developer patches were included in the original training data. To address this, we examined all the patches generated in our study and by SRepair$_{500}$ in the Defects4J dataset. Among the total plausible patches produced in our study, only 7.4‰ are identical to the developer patches. Similarly, for the plausible patches generated by SRepair$_{500}$, only 1.5‰ match the developer patches. Such overlapped patches pose almost no impact on our experimental results.

An additional threat lies in the trigger tests adopted in SRepair where the LLMs might have recognized the trigger tests and manipulated them to pass all tests, creating seemingly plausible patches. Our SRepair's Dual-LLM mechanism effectively mitigates this threat, as the repair suggestion model only suggests bug fixes without trigger test information, keeping the patch generation model isolated from such data.

Threats to external validity. The main threat to external validity lies in our evaluation datasets used which may not well generalize our experimental results. To mitigate this, we evaluate our approach on both the popular Defects4J 1.2 and 2.0 datasets where we include all their single-function bugs in our study. Furthermore, we extend our investigation to multi-function bugs in our SRepair evaluation. We also evaluate SRepair on the QuixBugs datasets, which contain both Java and Python bugs, to validate its generalizability.

Threats to construct validity. The threat to construct validity mainly lies in the metrics used. To mitigate this, we adopt the widely-used plausible patches along with their distributions. We also use correct fix to evaluate our approach SRepair.

7 CONCLUSION

In this paper, we conduct the first comprehensive study on the function-level LLM-based APR. Our study reveals that LLMs with zero-shot learning are powerful function-level APR techniques. Moreover, directly applying the auxiliary repair-relevant information to LLMs significantly increases the function-level repair performance. Inspired by our findings, we design a Dual-LLM framework utilizing Chain of Thought technique, named SRepair, which achieves remarkable repair performance by correctly fixing 300 single-function bugs in the Defects4J dataset, surpassing ChatRepair [79] by 85% and Repilot [74] by 1.59×. Notably, SRepair successfully fixes 32 multi-function bugs, which is the first time achieved by any APR technique ever to our best knowledge.

DATA AVAILABILITY

The data and code are available at *GitHub* [1] for public evaluation.

REFERENCES

[1] 2024. Github Repository. https://github.com/GhabiX/SRepair.
[2] 2024-02-29. Hugging Face. https://huggingface.co.
[3] 2024-02-29. OpenAI API. https://openai.com/api.
[4] 2024-03. Defects4J-GitHub. https://github.com/rjust/defects4j.
[5] 2024-03. Junit-5. https://junit.org/junit5/.
[6] 2024-03-04. CodeLlama huggingface. https://huggingface.co/codellama.
[7] 2024-03-04. ise-uiuc/magicoder. https://huggingface.co/ise-uiuc/Magicoder-S-CL-7B.
[8] 2024-03-04. New GPT-3 capabilities: Edit & insert. https://openai.com/blog/gpt-3-edit-insert.
[9] 2024-03-17. Defects4j Cli-26. https://github.com/rjust/defects4j/blob/master/framework/projects/Cli/patches/26.src.patch.
[10] 2024-03-17. Defects4j Closure-112. https://github.com/rjust/defects4j/blob/master/framework/projects/Closure/patches/112.src.patch.
[11] 2024-03-17. Defects4j Closure-66. https://github.com/rjust/defects4j/blob/master/framework/projects/Closure/patches/66.src.patch.
[12] 2024-03-17. Defects4j GitHub Repository. https://github.com/rjust/defects4j/.
[13] 2024-03-17. Defects4j JacksonDatabind-69. https://github.com/rjust/defects4j/blob/ master/framework/projects/JacksonDatabind/patches/69.src.patch.
[14] 2024-03-17. Defects4j Math-2. https://github.com/rjust/defects4j/blob/master/framework/projects/Math/patches/2.src.patch.
[15] 2024-03-17. Defects4j Math-80. https://github.com/rjust/defects4j/blob/master/framework/projects/Math/patches/80.src.patch.
[16] 2024-03-17. Defects4j Math-91. https://github.com/rjust/defects4j/blob/master/framework/projects/Math/patches/91.src.patch.
[17] 2024-03-17. Defects4j Math-95. https://github.com/rjust/defects4j/blob/master/framework/projects/Math/patches/95.src.patch.
[18] 2024-03-17. Flagship Model gpt-3.5-turbo-1106. https://platform.openai.com/docs/models/gpt-3-5-turbo.
[19] 2024-03-17. Implementation of multiple example selection. https://zenodo.org/records/7592886/files/patch.tar.gz?download=1.
[20] Monica Agrawal, Stefan Hegselmann, Hunter Lang, Yoon Kim, and David Sontag. 2022. Large language models are few-shot clinical information extractors. *arXiv preprint arXiv:2205.12689* (2022).
[21] Gabin An and Shin Yoo. 2022. FDG: a precise measurement of fault diagnosability gain of test cases. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 14–26.
[22] John Anvik, Lyndon Hiew, and Gail C Murphy. 2005. Coping with an open bug repository. In *Proceedings of the 2005 OOPSLA workshop on Eclipse technology eXchange*. 35–39.
[23] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models.
[24] Benoit Baudry, Franck Fleurey, and Yves Le Traon. 2006. Improving test suites for efficient fault localization. In *Proceedings of the 28th international conference on Software engineering*. 82–91.
[25] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What makes a good bug report?. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of software engineering*. 308–318.
[26] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, and et al. Kaplan. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, Vol. 33. 1877–1901.
[27] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting missing information in bug descriptions. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 396–407.
[28] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé De Oliveira Pinto, and et al Kaplan. 2021. Evaluating large language models trained on code. *arXiv (Cornell University)* (July 2021). https://doi.org/10.48550/arxiv.2107.03374
[29] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2023. Longlora: Efficient fine-tuning of long-context large language models. *arXiv preprint arXiv:2309.12307* (2023).
[30] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. Sequencer: Sequence-to-sequence learning for end-to-end program repair. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1943–1959.
[31] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. 423–435.
[32] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2023. Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzgpt. *arXiv preprint arXiv:2304.02014* (2023).
[33] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481.
[34] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2018. Automatic software repair: A survey. In *Proceedings of the 40th International Conference on Software Engineering*. 1219–1219.
[35] Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish Shevade. 2017. Deepfix: Fixing common c language errors by deep learning. In *Proceedings of the aaai conference on artificial intelligence*, Vol. 31.
[36] Mark Harman. 2010. Automated patching techniques: the fix is in: technical perspective. *Commun. ACM* 53, 5 (2010), 108–108.
[37] Kai Huang, Xiangxin Meng, Jian Zhang, Yang Liu, Wenjie Wang, Shuhao Li, and Yuqing Zhang. 2023. An empirical study on fine-tuning large language models of code for automated program repair. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1162–1174.
[38] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. *arXiv preprint arXiv:2302.05020* (2023).
[39] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. Cure: Code-aware neural machine translation for automatic program repair. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1161–1173.
[40] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. Inferfix: End-to-end program repair with llms. *arXiv preprint arXiv:2303.07263* (2023).
[41] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.
[42] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.
[43] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic patch generation learned from human-written patches. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 802–811.
[44] Pavneet Singh Kochhar, Xin Xia, David Lo, and Shanping Li. 2016. Practitioners' expectations on automated fault localization. In *Proceedings of the 25th international symposium on software testing and analysis*. 165–176.
[45] Sophia D Kolak, Ruben Martins, Claire Le Goues, and Vincent Josua Hellendoorn. 2022. Patch generation with language models: Feasibility and scaling behavior. In *Deep Learning for Code Workshop*.
[46] Anil Koyuncu, Kui Liu, Tegawendé F Bissyandé, Dongsun Kim, Martin Monperrus, Jacques Klein, and Yves Le Traon. 2019. iFixR: Bug report driven program repair. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 314–325.
[47] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 919–931.
[48] Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. 2019. Deepfl: Integrating multiple fault diagnosis dimensions for deep fault localization. In *Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis*. 169–180.
[49] Yi Li, Shaohua Wang, and Tien N Nguyen. 2020. Dlfix: Context-based code transformation learning for automated program repair. In *Proceedings of the ACM/IEEE 42nd international conference on software engineering*. 602–614.
[50] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. QuixBugs: A multi-lingual program repair benchmark set based on the Quixey Challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN international conference on systems, programming, languages, and applications: software for humanity*. 55–56.
[51] Chen Liu, Jinqiu Yang, Lin Tan, and Munawar Hafiz. 2013. R2Fix: Automatically generating bug fixes from bug reports. In *2013 IEEE Sixth international conference on software testing, verification and validation*. IEEE, 282–291.
[52] Kui Liu, Anil Koyuncu, Tegawendé F Bissyandé, Dongsun Kim, Jacques Klein, and Yves Le Traon. 2019. You cannot fix what you cannot find! an investigation of fault localization bias in benchmarking automated program repair systems. In *2019 12th IEEE conference on software testing, validation and verification (ICST)*. IEEE, 102–113.
[53] Fan Long and Martin Rinard. 2015. Staged program repair with condition synthesis. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 166–178.
[54] Fan Long and Martin Rinard. 2016. An analysis of the search spaces for generate and validate patch generation systems. In *Proceedings of the 38th International Conference on Software Engineering*. 702–713.
[55] Yiling Lou, Ali Ghanbari, Xia Li, Lingming Zhang, Haotian Zhang, Dan Hao, and Lu Zhang. 2020. Can automated program repair refine fault localization? a unified debugging approach. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 75–87.

[56] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. Coconut: combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*. 101–114.

[57] Matias Martinez, Thomas Durieux, Romain Sommerard, Jifeng Xuan, and Martin Monperrus. 2017. Automatic repair of real bugs in java: A large-scale experiment on the defects4j dataset. *Empirical Software Engineering* 22 (2017), 1936–1964.

[58] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th international conference on software engineering*. 691–701.

[59] Ali Mesbah, Andrew Rice, Emily Johnston, Nick Glorioso, and Edward Aftandilian. 2019. Deepdelta: learning to repair compilation errors. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 925–936.

[60] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. Semfix: Program repair via semantic analysis. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 772–781.

[61] Fabio Petroni, Tim Rocktäschel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, Alexander H Miller, and Sebastian Riedel. 2019. Language models as knowledge bases? *arXiv preprint arXiv:1909.01066* (2019).

[62] Julian Aron Prenner, Hlib Babii, and Romain Robbes. 2022. Can OpenAI's codex fix bugs? an evaluation on QuixBugs. In *Proceedings of the Third International Workshop on Automated Program Repair*. 69–75.

[63] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, and et al. Sootla. 2023. Code llama: Open Foundation Models for code. *arXiv (Cornell University)* (Aug. 2023). https://doi.org/10.48550/arxiv.2308.12950

[64] Ripon K Saha, Yingjun Lyu, Hiroaki Yoshida, and Mukul R Prasad. 2017. Elixir: Effective object-oriented program repair. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 648–659.

[65] Victor Sobreira, Thomas Durieux, Fernanda Madeiral, Martin Monperrus, and Marcelo de Almeida Maia. 2018. Dissection of a bug dataset: Anatomy of 395 patches from defects4j. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 130–140.

[66] Ezekiel Soremekun, Lukas Kirschner, Marcel Böhme, and Mike Papadakis. 2023. Evaluating the impact of experimental assumptions in automated fault localization. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 159–171.

[67] Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. 2019. How to fine-tune bert for text classification?. In *Chinese computational linguistics: 18th China national conference, CCL 2019, Kunming, China, October 18–20, 2019, proceedings 18*. Springer, 194–206.

[68] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to Sequence Learning with Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 27.

[69] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLAMA: Open and Efficient Foundation Language Models. *arXiv (Cornell University)* (Feb. 2023). https://doi.org/10.48550/arxiv.2302.13971

[70] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) *(NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 6000–6010.

[71] Weishi Wang, Yue Wang, Shafiq Joty, and Steven CH Hoi. 2023. Rap-gen: Retrieval-augmented patch generation with codet5 for automatic program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 146–158.

[72] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[73] Yuxiang Wei, Zhe Wang, Jiawei Liu, Ye Ding, and Lingming Zhang. 2023. MagicOder: Source code is all you need. *arXiv (Cornell University)* (Dec. 2023). https://doi.org/10.48550/arxiv.2312.02120

[74] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 172–184.

[75] Chunqiu Steven Xia, Yifeng Ding, and Lingming Zhang. 2023. Revisiting the Plastic Surgery Hypothesis via Large Language Models. *arXiv preprint arXiv:2303.10494* (2023).

[76] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023). Association for Computing Machinery.*

[77] Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971.

[78] Chunqiu Steven Xia and Lingming Zhang. 2023. Conversational automated program repair. *arXiv preprint arXiv:2301.13246* (2023).

[79] Chunqiu Steven Xia and Lingming Zhang. 2023. Keep the Conversation Going: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. *arXiv preprint arXiv:2304.00385* (2023).

[80] Yingfei Xiong, Jie Wang, Runfa Yan, Jiachen Zhang, Shi Han, Gang Huang, and Lu Zhang. 2017. Precise condition synthesis for program repair. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 416–426.

[81] He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[82] He Ye, Matias Martinez, and Martin Monperrus. 2022. Neural program repair with execution-based backpropagation. In *Proceedings of the 44th International Conference on Software Engineering*. 1506–1518.

[83] He Ye and Martin Monperrus. 2024. ITER: Iterative Neural Repair for Multi-Location Patches. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[84] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. 2023. A survey on multimodal large language models. *arXiv preprint arXiv:2306.13549* (2023).

[85] Quanjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. Gamma: Revisiting template-based automated program repair via mask prediction. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 535–547.

[86] Quanjun Zhang, Tongke Zhang, Juan Zhai, Chunrong Fang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. A critical review of large language model on software engineering: An example from chatgpt and automated program repair. *arXiv preprint arXiv:2310.08879* (2023).

[87] Tao Zhang, Jiachi Chen, He Jiang, Xiapu Luo, and Xin Xia. 2017. Bug report enrichment with application of automated fixer recommendation. In *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*. IEEE, 230–240.

[88] Zhuosheng Zhang, Aston Zhang, Mu Li, Hai Zhao, George Karypis, and Alex Smola. 2023. Multimodal chain-of-thought reasoning in language models. *arXiv preprint arXiv:2302.00923* (2023).

[89] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 341–353.